

Description Accepted X Editorial Solutions Submissions

1971. Find if Path Exists in Graph

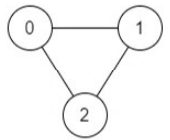
Easy Topics Companies

There is a **bi-directional** graph with n vertices, where each vertex is labeled from 0 to $n - 1$ (**inclusive**). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [ui, vi]` denotes a bi-directional edge between vertex u_i and vertex v_i . Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

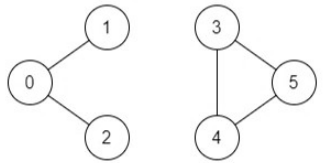
Given `edges` and the integers `n`, `source`, and `destination`, return `true` if there is a **valid path** from `source` to `destination`, or `false` otherwise.

Example 1:



Input: `n = 3, edges = [[0,1],[1,2],[2,0]], source = 0, destination = 2`
Output: `true`
Explanation: There are two paths from vertex 0 to vertex 2:
- `0 → 1 → 2`
- `0 → 2`

Example 2:



Input: `n = 6, edges = [[0,1],[0,2],[3,5],[5,4],[4,3]], source = 0, destination = 5`
Output: `false`
Explanation: There is no path from vertex 0 to vertex 5.

Constraints:

4.2K 233 ☆

Solved

0 Online

Code

C

```
1 bool validPath(int n, int** edges, int edgesSize, int* edgesColSize, int source, int destination) {
2     if (source == destination) return true;
3
4     int *deg = calloc(n, sizeof(int));
5     for (int i = 0; i < edgesSize; i++) {
6         deg[ edges[i][0] ]++;
7         deg[ edges[i][1] ]++;
8     }
9
10    int **adj = malloc(n * sizeof(int*));
11    for (int i = 0; i < n; i++) {
12        adj[i] = malloc(deg[i] * sizeof(int));
13        deg[i] = 0;
14    }
15
16    for (int i = 0; i < edgesSize; i++) {
17        int a = edges[i][0];
18        int b = edges[i][1];
19        adj[a][ deg[a]++ ] = b;
20        adj[b][ deg[b]++ ] = a;
21    }
22
23    bool *mark = calloc(n, sizeof(bool));
24    int *stk = malloc(n * sizeof(int));
25    int top = 0;
26
27    stk[top++] = source;
28    mark[source] = true;
29
30    while (top > 0) {
31        int u = stk[--top];
32        if (u == destination) return true;
33
34        for (int i = 0; i < deg[u]; i++) {
35            int v = adj[u][i];
36            if (!mark[v]) {
37                mark[v] = true;
38                stk[top++] = v;
39            }
40        }
41    }
42
43    return false;
44 }
```

Saved

Activate Windows
Go to Settings to activate Windows.

Ln 29, Col 1

Description Accepted X Editorial Solutions Submissions

All Submissions

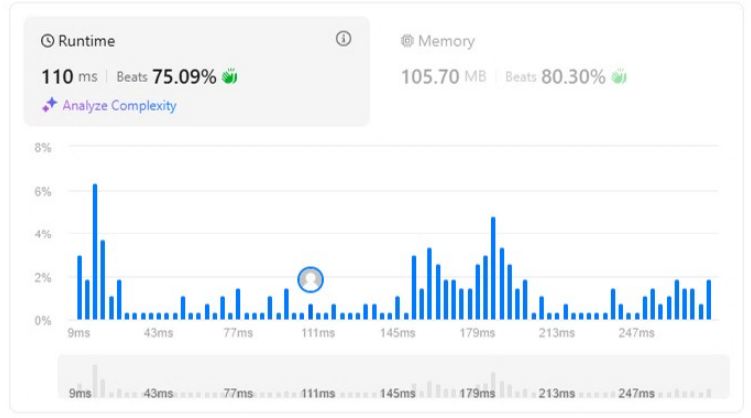
Accepted 34 / 34 testcases passed

monisha_r_ submitted at Dec 01, 2025 11:57

Editorial Solution

LIMITED TIME OFFER - \$40 off Annual Subscription

LeetCode's Thanksgiving Sale IS NOW LIVE! Get \$40 OFF - Your Final Offer of the Year! →



Code C

```
1 bool validPath(int n, int** edges, int edgesSize, int* edgesColSize, int sou
2     if (source == destination) return true;
3
4     int *deg = calloc(n, sizeof(int));
5     for (int i = 0; i < edgesSize; i++) {
6         deg[ edges[i][0] ]++;
7         deg[ edges[i][1] ]++;
8     }
```

View more

- More challenges
- 2097. Valid Arrangement of Pairs
 - 2077. Paths in Maze That Lead to Same Room

Write your notes here

```
1 bool validPath(int n, int** edges, int edgesSize, int* edgesColSize, int source, int destination) {
2     if (source == destination) return true;
3
4     int *deg = calloc(n, sizeof(int));
5     for (int i = 0; i < edgesSize; i++) {
6         deg[ edges[i][0] ]++;
7         deg[ edges[i][1] ]++;
8     }
9
10    int **adj = malloc(n * sizeof(int*));
11    for (int i = 0; i < n; i++) {
12        adj[i] = malloc(deg[i] * sizeof(int));
13        deg[i] = 0;
14    }
15
16    for (int i = 0; i < edgesSize; i++) {
17        int a = edges[i][0];
18        int b = edges[i][1];
19        adj[a][ deg[a]++ ] = b;
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n = 3

edges = [[0,1],[1,2],[2,0]]

source = 0

destination = 2

Output

true

Activate Windows Go to Settings to activate Windows.